

KASPARRO

AI Representation Optimizer for Shopify

Product Document | Kasparro Agentic Commerce Hackathon | April 2026

Prepared by Aswin Kumar B S and Naveen D

Kasparro tells Shopify merchants exactly why AI shopping assistants are ignoring their products — and fixes it automatically, in one click.

Category	Shopify SaaS — AI content intelligence and optimization
Primary user	Shopify merchants \$100K-\$5M ARR with no dedicated SEO or content team
Modules built	16 AI-powered analysis and optimization modules
AI providers	Gemini 2.0 Flash, OpenRouter, Groq, AWS Bedrock, GPT-4o-mini, Tavily, SerpAPI
Infrastructure	Vercel + Railway + Neon PostgreSQL

1. The Problem We Are Solving

Product discovery is migrating from keyword search to AI-generated answers. ChatGPT Shopping, Google AI Overviews, and Perplexity now synthesise product recommendations directly — no click required, no result page to rank on. A product whose content an AI cannot parse and cite simply does not appear in that buying journey.

This matters because the signals that make a product AI-visible are completely different from traditional SEO signals. AI agents do not care about backlink counts or page speed. They care about whether a product description contains structured specifications, unambiguous benefit statements, FAQ coverage, schema markup, and consistent brand voice. None of the tools Shopify merchants currently have — Ahrefs, SEMrush, or Shopify's own analytics — measure any of these signals.

The result: merchants whose products are being systematically excluded from AI-generated buying recommendations have no way of knowing it is happening, no way of diagnosing why, and no way to fix it. They keep running ads against a discovery surface that is shrinking while a parallel AI-driven surface — where their competitors are appearing — grows invisible to their existing tooling.

"ChatGPT is recommending products in my category every day. Why not mine — and what exactly do I change?"

No tool in the Shopify ecosystem answers this question today. Kasparro does.

Who We Are Building For and Their Current Experience

Persona	Current Experience Without Kasparro
Mid-market Shopify merchant \$100K-\$5M ARR, 2-5 person team	Adds products, runs ads, checks Google Analytics. Sees conversion drop and suspects the content but has no evidence. Does not know competitors restructured their descriptions 6 months ago and now appear in ChatGPT Shopping while their products do not. If they want to test AI visibility, they manually copy-paste product descriptions into ChatGPT and guess from the output.
Shopify agency or consultant Managing 5-50 stores	Knows their clients need to be AI-ready but has no repeatable audit process to prove it, no benchmarks to set client expectations against, and no before/after data to demonstrate the value of their optimisation work. Presents qualitative recommendations that clients cannot verify.

2. Core User Journey

This is what a merchant actually does from first login to seeing measurable improvement.

Step	What Happens	What the Merchant Sees / Decides
1. Connect	Google OAuth login, then Shopify OAuth installs the app and stores an AES-256-GCM encrypted access token	One-time setup; no manual API key entry; takes under 2 minutes
2. Sync catalog	POST /products/sync fetches full Shopify catalog via Admin REST API; upserts all products to Neon DB	Progress indicator; product count confirmed; ready to analyse
3. First analysis	Analysis pipeline scores every product across 5 dimensions using ~40 rules + Gemini AI; gaps and fixes pre-generated in the same pass	Dashboard shows store aggregate score (e.g. 61/100) and per-product breakdown; red/amber/green signals make priorities obvious
4. Read the gaps	Gap cards show exact rule violations with evidence: 'Title has 2 words — minimum 6 required (impact: high)'	Merchant sees specifically what is wrong on each product, ranked by impact score
5. Apply a fix	Merchant clicks Apply on a fix; optionally edits the AI-generated content; API writes to Shopify Admin REST, reads back the updated field, updates the product score only if read-back confirms	One-click resolution; score updates immediately on confirmed sync; no guesswork about whether it landed
6. Run GEO scan	POST /geo/scan fires real queries against Tavily, SerpAPI, Bedrock, GPT-4o-mini, and Gemini; results persist as citation records	Merchant sees which AI platforms cite them, which do not, and which queries they are missing from — with competitor names appearing in their place

Step	What Happens	What the Merchant Sees / Decides
7. Schedule + monitor	Weekly or monthly automated analysis; score snapshots appended after each run; email sent on regression or when score changes by a configured delta	Merchant does nothing; gets emailed if their score drops; can see the trend chart over time

3. Key Product Decisions and the Reasoning Behind Each

Decision 1: Shopify-Only, Not Multi-Platform

What we chose: Build exclusively for Shopify merchants in this version. BigCommerce, WooCommerce, and Wix are explicitly out of scope.

Why: Shopify has a single, stable Admin REST API that returns consistent product data structures across all stores. Building a multi-platform normalisation layer would have consumed 40% of the available build time for no additional insight value in the hackathon context. More importantly, Shopify has a well-defined App Store distribution channel with 1.7M merchants — the beachhead is large enough to prove the model before expanding. Every feature (webhook sync, fix deployment, LLMs.txt deploy) is tightly coupled to Shopify's API surface. Abstracting it prematurely would have made all three features brittle.

Decision 2: 60/40 Rule Engine + AI Blend, Not Pure AI

What we chose: Each dimension score is 60% deterministic rule engine + 40% Gemini quality assessment.

Why: Pure AI scoring is non-deterministic — the same product can score differently across runs, making trend tracking meaningless. Pure rule scoring misses nuance — a technically valid description ('Has 8 words, no passive voice, includes brand name') can still be weak content that an AI agent would not cite. The blend gives us auditability (merchant can see which rule fired) and quality sensitivity (AI catches what rules miss). The 60/40 split was chosen after testing: below 50% rules the scores drifted too much between runs; above 70% rules the AI contribution was too small to catch genuine quality differences.

Decision 3: Pre-Generate Fixes During Analysis, Not On-Demand

What we chose: AI-generated fix content is produced for all high-severity gaps during the analysis pipeline pass, stored immediately, and served instantly on the dashboard.

Why: The alternative — generating fixes on-demand when the merchant clicks 'Fix' — would mean a 3-8 second Gemini latency at the moment of merchant action, breaking the one-click feel. Pre-generating means

the fix is ready before the merchant needs it. The cost is higher storage and occasional waste (merchant may not apply all fixes). We accepted this tradeoff because the UX improvement is significant and the storage cost is negligible.

Decision 4: Write Fixes to Shopify + Read-Back Verify, Not Just Show Suggestions

What we chose: Kasparro writes approved fixes directly to the merchant's Shopify store via the Admin API, reads back the updated field, and only credits the score improvement if the read-back confirms the value landed.

Why: Tools that only suggest changes leave the loop open — merchants must copy text, navigate to Shopify, paste, save, come back and mark it done. Closing the loop in one click is the core value proposition. The read-back verification was added after we noticed that Shopify API write calls return 200 even when the content is truncated or the update is silently throttled. Without read-back, we would have shown score improvements that did not actually land. The read-back adds one extra API call per fix but makes the score data trustworthy.

Decision 5: 5-Platform GEO Scan, Not a Single Engine

What we chose: The GEO scan fires real queries against five independent platforms: Tavily (web citations), SerpAPI (Google organic), AWS Bedrock Nova (brand knowledge), OpenAI GPT-4o-mini (content quality), and Gemini (consistency).

Why: A single AI engine's citation is an anecdote. A store could be cited by GPT-4o but invisible to Google AI Overviews because it does not rank organically for any relevant query. Each platform tests a different layer of AI visibility: web presence, organic ranking, brand knowledge, content quality, and answer consistency. A store that scores well on all five has robust, multi-surface AI discoverability. A store that scores well on only one has a fragile presence that could disappear with one algorithm change.

4. What We Chose NOT to Build and Why

Scope Decision	Reasoning
Direct Shopify theme injection for JSON-LD schema	Injecting code into a merchant's theme without their review is high-risk — a bad inject can break the storefront. We generate the JSON-LD and show step-by-step instructions instead. The merchant applies it with awareness. Closing this last 10% of automation is on the roadmap but requires Shopify theme API access we did not build yet.
Real-time product monitoring (continuous webhooks)	Webhook processing is implemented (products-update, products-delete) but we do not re-score on every webhook. Re-scoring on every save would burn Gemini quota rapidly for active merchants. Batch analysis on a schedule is the right cost/value tradeoff at this stage.

Scope Decision	Reasoning
Custom AI model training on merchant data	Training a domain-specific model would take months and require GDPR consent architecture we did not have time to design properly. The prompt engineering approach with Gemini delivers 85-90% of the value of a fine-tuned model with no data custody risk.
B2B / wholesale merchant support	B2B merchants selling to procurement teams have fundamentally different AI discovery patterns — they are optimised for specification-heavy comparison queries, not consumer intent questions. A different scoring model is needed. Scope explicitly excluded to avoid diluting the B2C scoring logic.
Mobile app	Store management happens on desktop. Shopify's own data shows under 8% of merchant admin activity is mobile. Building a mobile app would have consumed build time that went into the GEO scanner and competitor intelligence instead.
Multi-language content analysis	Our rule engine fires against English-language content. Multi-language support requires locale-aware NLP rules and multilingual Gemini prompts. Excluded for this build; marked as future roadmap item.

5. Tradeoffs We Encountered and How We Resolved Them

Tradeoff 1: Breadth vs. Depth

We built 16 modules rather than going 3 modules deep. The reasoning: at this stage of the product, the breadth of surface area is evidence of how much of the AI-readiness problem Kasparro owns. A merchant who runs GEO scan, fixes descriptions, deploys LLMs.txt, and sets up a schedule is significantly more AI-ready than one who only fixed their descriptions. Breadth also surfaces the product's scope to judges and investors more clearly. The risk of breadth is that no single module is exhaustive. We accepted this and designed each module to be the best first version, not the final version.

Tradeoff 2: Automation vs. Merchant Control in Fix Application

We could have made Kasparro fully automatic — detect a gap, apply the fix, no user action required. We chose not to. Merchants have strong feelings about their brand voice and product copy. An unexpected change to a product description — even a better one — destroys trust. The edit-before-apply flow gives merchants enough control that they feel safe giving Kasparro Admin API write access. We see this as the right balance: Kasparro does the analysis and writes the content, but the merchant authorises every write. The bulk-apply feature lets power users move fast when they trust the output.

Tradeoff 3: In-Process Queue vs. External Job Queue

The analysis queue is an in-process FIFO queue (`maxConcurrent=1`), not a Redis + Bull external queue. The tradeoff: in-process is simpler, zero infrastructure cost, and works reliably on a single Railway instance. It does not survive server restarts mid-analysis. We mitigated this with a jobs table and idempotency keys — if the server restarts, the client can re-POST with the same idempotency key and get the existing job back. For a hackathon build serving tens of stores, in-process is the right call. At 1,000 concurrent stores, Redis + Bull becomes necessary.

Tradeoff 4: Rule Count vs. False Positive Rate

The rule engine has approximately 40 rules. We deliberately erred on the side of more gaps reported rather than fewer. A false positive (flagging something that is actually fine) is less harmful than a false negative (missing a genuine problem). Merchants can dismiss a gap they disagree with. They cannot fix a problem they never knew existed. The risk is alert fatigue on large catalogs. We mitigated this with impact scores and severity tiers — high severity gaps are shown first, giving merchants a clear action hierarchy even when many gaps are present.

What We Would Do Differently with More Time

- Replace the in-process analysis queue with Bull + Redis for horizontal scalability and crash resilience
- Build a proper fix suggestion review UI — right now the edit field is plain text; a rich-text diff editor would make reviewing AI-generated content much faster
- Add a proper A/B testing layer to the GEO scanner so merchants can measure the before/after citation rate change from a specific fix, not just overall score trend
- The 60/40 blend ratio was set empirically; with more time we would run a structured evaluation against a labeled dataset of product descriptions to calibrate the optimal blend ratio per industry vertical